

Calidad de software: evaluación, aprendizaje y mejora continua

Breve descripción:

Este componente desarrolla la evaluación de la calidad del software, mediante control de calidad, herramientas de pruebas y documentación de hallazgos. Incluye la verificación de requisitos no funcionales como seguridad, desempeño y confiabilidad, así como la gestión de lecciones aprendidas y la formulación de planes de mejora con acciones correctivas, preventivas y seguimiento.

Mayo 2026

Tabla de contenido

Introducción	4
1. Evaluación de calidad del software	7
1.1. Control de calidad.....	7
1.2. Herramientas de evaluación.....	8
1.3. Documentación de hallazgos.....	15
2. Evaluación de requisitos no funcionales	19
3. Gestión del conocimiento en el desarrollo de software	25
3.1. Modelos de gestión del conocimiento	26
3.2. Lecciones aprendidas en proyectos de software	28
3.3. Recomendaciones de buenas prácticas.....	32
3.4. Herramientas para la gestión del conocimiento.....	35
4.1. Elaboración del plan de mejora	39
4.2. Acciones correctivas	43
4.3. Acciones preventivas	45
4.4. Responsables del plan de mejora	48
4.5. Verificación y seguimiento.....	52
Síntesis	55
Glosario	57

Referencias bibliográficas	59
Créditos	60

Introducción

La evaluación de la calidad del software constituye una etapa fundamental dentro del ciclo de vida del desarrollo, ya que permite verificar que el producto entregado cumple con los requisitos establecidos y responde adecuadamente a las necesidades del usuario y del entorno operativo. En un contexto donde los sistemas informáticos soportan procesos críticos de negocio, la calidad no puede limitarse únicamente a la funcionalidad; debe abarcar atributos como seguridad, desempeño, confiabilidad y adaptabilidad. Por ello, este componente formativo aborda el control de calidad como un proceso técnico estructurado que integra herramientas especializadas, análisis de métricas y documentación rigurosa de hallazgos, garantizando trazabilidad y soporte para la toma de decisiones.

Asimismo, el profundiza en la evaluación de requisitos no funcionales — seguridad, usabilidad, accesibilidad, portabilidad, tiempos de respuesta, adaptabilidad y confiabilidad— entendidos como factores determinantes para la estabilidad y sostenibilidad del software en producción. Complementariamente, se incorpora la gestión del conocimiento mediante la sistematización de lecciones aprendidas y recomendaciones técnicas y se establecen lineamientos para la formulación de planes de mejora que incluyan acciones correctivas, preventivas y de optimización, con responsables definidos y mecanismos de verificación y seguimiento. De esta manera, se consolida una visión integral de la calidad orientada no solo a detectar fallas, sino a fortalecer continuamente los procesos y el servicio de software.

Para entender de mejor manera estas acciones iniciales, se invita a que acceda al siguiente video, el cual relaciona la temática a tratar durante este componente formativo:

Video 1. Calidad de software: evaluación, aprendizaje y mejora continua



[Enlace de reproducción del video](#)

Transcripción del video: Calidad de software: evaluación, aprendizaje y mejora continua

En el desarrollo de software, la calidad no es un resultado aislado, sino un proceso continuo de evaluación y mejora que acompaña cada etapa del ciclo de vida. Por este motivo, este componente formativo inicia con el estudio de la evaluación de la calidad del software, entendida como un mecanismo técnico para verificar que las soluciones cumplen con los requisitos definidos y responden de manera efectiva a las necesidades del usuario y del entorno operativo.

A lo largo del recorrido, se aborda el control de calidad como un proceso estructurado, apoyado en herramientas de evaluación, métricas técnicas y documentación rigurosa de hallazgos. De forma complementaria, se profundiza en la evaluación de los requisitos no funcionales, como la seguridad, el desempeño, la accesibilidad, la usabilidad y la confiabilidad, reconociendo su impacto directo en la estabilidad y sostenibilidad del software en producción.

El componente integra además la gestión del conocimiento en el desarrollo de software, promoviendo la sistematización de lecciones aprendidas, la transferencia de buenas prácticas y el uso de herramientas que permiten preservar y reutilizar el conocimiento técnico generado en los proyectos. Finalmente, se consolidan estos aprendizajes mediante la formulación de planes de mejora, que articulan acciones correctivas y preventivas, responsables definidos y mecanismos de verificación y seguimiento.

De esta manera, se construye una visión integral de la calidad, orientada no solo a detectar fallas, sino a fortalecer continuamente los procesos, los equipos y el servicio de software, asegurando soluciones confiables, sostenibles y alineadas con los objetivos organizacionales.

1. Evaluación de calidad del software

La calidad del software es el grado en que un sistema satisface los requisitos especificados y las necesidades implícitas del usuario. Se evalúa desde múltiples dimensiones usando modelos estandarizados internacionalmente.

1.1. Control de calidad

Es el proceso sistemático de verificar que un producto final — ya sea un reporte automatizado, un dashboard o una macro — cumpla con las especificaciones técnicas y las necesidades reales del negocio.

La gestión de la calidad del software se fundamenta en tres pilares esenciales:

- **Calidad de producto**

Evalúa atributos internos y externos como funcionalidad, rendimiento y seguridad mediante mediciones estáticas y dinámicas.

- **Calidad en uso**

Centrada en la experiencia del usuario final, respecto a la efectividad y satisfacción en su contexto real.

- **Calidad del proceso**

Asegura la madurez del desarrollo, mediante modelos como CMMI o ISO 15504 (SPICE).

Esta estructura integral garantiza que un proceso bien ejecutado, derive naturalmente en un producto de alto nivel que cumpla con las expectativas operativas y técnicas.

Para estandarizar estas mediciones, se emplea la serie ISO/IEC 25000 (SQuaRE), que establece los marcos de referencia actuales para métricas y requisitos. Es crucial diferenciar entre el testing, enfocado en la corrección funcional y la evaluación, un concepto más amplio que abarca auditorías y análisis de mantenibilidad. Todo este ecosistema se apoya en estándares clave como la ISO 25010 para modelos de calidad, la ISO 25040 para procesos de evaluación, la IEEE 730 para el aseguramiento y la ISO 9001 para la gestión de calidad global.

1.2. Herramientas de evaluación

Existen distintas perspectivas y momentos para evaluar la calidad. Lo ideal es combinarlos durante todo el ciclo de vida del software; para ello existen diferentes enfoques y herramientas de evaluación. Por consiguiente, para optimizar la calidad del software, se emplean diversas técnicas complementarias:

- **Análisis estático**

Permite examinar el código sin ejecutarlo para detectar deuda técnica y malas prácticas mediante herramientas como SonarQube o ESLint.

- **Análisis dinámico**

Utiliza casos de prueba (unitarias, integración y sistema) para medir la cobertura de código en ejecución.

Este proceso se fortalece con la revisión entre pares, una inspección colaborativa sumamente eficaz para la detección temprana de errores y se consolida a través de la integración continua (CI), que automatiza las pruebas y el análisis de calidad en cada commit, utilizando pipelines en plataformas como Jenkins o GitHub Actions.

- **Herramientas de pruebas funcionales**

Las pruebas funcionales se centran en verificar "qué" hace el sistema. Su objetivo es confirmar que cada función de la solución de software opere de acuerdo con las especificaciones del cliente o los resultados de aprendizaje esperados. A partir de este enfoque, las herramientas de pruebas funcionales se organizan en tres ejes fundamentales que permiten evaluar el comportamiento del sistema de manera integral:

- **Validación de entradas y restricciones**

Constituye la primera línea de defensa para garantizar la calidad funcional, utilizando la validación de datos nativa para restringir ingresos erróneos y listas desplegables que aseguran la estandarización de categorías. Complementariamente, las pruebas de integridad en el modelado (caja negra) verifican el flujo de información externa, mediante el cruce de información con funciones como BUSCARV/XLOOKUP y el uso de tablas de Excel (objetos), lo que permite que las referencias sean dinámicas y hereden automáticamente las reglas funcionales.

- **Procesamiento**

En este eje las herramientas de cálculo y lógica automática aseguran que las columnas calculadas y las funciones de texto y fecha devuelvan valores esperados bajo distintos escenarios; por otro lado, las tablas dinámicas y segmentadores (slicers)

actúan como un "test de estrés" funcional para la visualización, permitiendo la agrupación y filtrado de datos y una segmentación interactiva que facilita a la gerencia filtrar información en tiempo real, validando así la respuesta del sistema ante las preguntas del negocio.

- **Grabadora de macros y automatización**

Se utiliza para la generación de "scripts" funcionales, donde la prueba definitiva consiste en la ejecución de macros para validar que el informe final coincida con el plan de acción. Para mejorar la interfaz de usuario (botones), estas automatizaciones se vinculan a un "panel de control", garantizando que la funcionalidad sea accesible para usuarios sin conocimientos técnicos y asegurando una experiencia de uso fluida y profesional.

Métrica de éxito: una prueba funcional es exitosa si el analista logra transformar datasets desorganizados en un dashboard interactivo que responda con exactitud a los requerimientos gerenciales en el tiempo estipulado.

- **Herramientas de pruebas no funcionales**

Mientras que las pruebas funcionales se centran en el "qué" hace el software, las pruebas no funcionales evalúan el "cómo" lo hace. En el contexto de Excel avanzado, estas herramientas miden la robustez, el rendimiento y la experiencia del usuario (UX) frente a grandes volúmenes de datos.

Las pruebas de rendimiento e irrupción evalúan la estabilidad de la hoja de cálculo ante el crecimiento de la información. Se enfocan en la gestión de grandes volúmenes de datos y la optimización de fórmulas para evitar lentitud. Además, miden

el tiempo de ejecución de macros para validar el ahorro de tiempo real frente a procesos manuales.

En cuanto a las pruebas de usabilidad e interfaz de usuario (UX), estas garantizan que herramientas como el diseño de dashboards y la navegación interactiva con slicers sean fluidas. Es fundamental que el panel de control cuente con botones de la pestaña "programador" que tengan nombres claros para facilitar el trabajo de los destinatarios finales.

Las pruebas de fiabilidad y consistencia aseguran que el sistema sea resistente a fallos, mediante la validación de la macro, realizando tests de "borrado y ejecución" para obtener resultados constantes. La documentación y planificación previa del algoritmo es una herramienta no funcional clave para reducir drásticamente el riesgo de errores operativos a largo plazo.

Para garantizar la integridad en entornos empresariales, las pruebas de seguridad y protección resultan fundamentales para resguardar la propiedad intelectual y el código. Esto incluye la protección de estructura en hojas y libros para evitar que terceros alteren el modelado, sumado a una correcta configuración de la seguridad de macros que restrinja su ejecución a entornos de confianza, previniendo así cualquier riesgo informático.

A continuación, se sintetizan los principales atributos evaluados en las pruebas no funcionales, junto con las herramientas o técnicas empleadas y su objetivo dentro del programa:

Tabla 1. Atributos y herramientas de evaluación en pruebas no funcionales

Atributo	Herramienta / técnica	Objetivo en el programa
Eficiencia	Cronometrar ejecución de macros.	Reducir tiempos de 4 horas a minutos.
Escalabilidad	Modelado con tablas oficiales.	Soportar crecimiento rápido de transacciones.
Usabilidad	Dashboard con segmentadores.	Facilitar la toma de decisiones gerenciales.

- **Herramientas de seguimiento de defectos**

En la ingeniería de software, un defecto es cualquier desviación entre el comportamiento esperado y el comportamiento real del sistema; es por ello, que el seguimiento de defectos es el proceso de identificar, registrar y priorizar estos fallos para asegurar la calidad del producto final.

Por su parte, el proceso de depuración (debugging) es el conjunto de técnicas fundamentales para encontrar la causa raíz de un fallo, mediante tres etapas clave: la identificación del comportamiento anómalo, el aislamiento del módulo o línea de código origen y la corrección lógica para evitar la regresión. Este ciclo se apoya en el seguimiento de defectos, un enfoque reactivo que permite el registro y la priorización de errores, según su impacto, desde fallos críticos en cálculos hasta detalles cosméticos, garantizando que cada desviación sea validada tras su reparación.

Para ejecutar estas tareas, se utilizan herramientas técnicas de depuración como la ejecución paso a paso, los puntos de interrupción (breakpoints) y la inspección de variables (locals/watch), que permiten monitorear el flujo de datos en tiempo real. Además, la robustez del sistema se refuerza con la gestión de excepciones, empleando bloques de manejo de errores (error handling) para capturar fallos antes del colapso,

así como el control de eventos, que supervisa acciones inesperadas del sistema para asegurar una respuesta controlada y profesional.

- **Herramientas de análisis y métricas**

En la ingeniería de software, "lo que no se mide, no se puede mejorar". Las métricas proporcionan datos objetivos para evaluar la eficacia del proceso de desarrollo y la calidad del producto final.

Las métricas de defectos son fundamentales para cuantificar la calidad interna del software, destacando indicadores como la densidad de defectos (errores por módulo o líneas de código) y la tasa de fallos, que mide la frecuencia de errores en un tiempo determinado. Para evaluar la capacidad de respuesta del equipo, se utiliza el tiempo medio de reparación (MTTR), el cual promedia el tiempo invertido en identificar, depurar y corregir cada bug. Estas cifras permiten un control riguroso sobre la estabilidad técnica de la herramienta antes de su despliegue definitivo.

Por otro lado, la calidad externa se mide a través de métricas de eficiencia y rendimiento, las cuales reflejan el impacto real en el negocio, mediante el ahorro de tiempo operativo y la notable reducción del error humano tras la estandarización de procesos. Para visualizar estos resultados, se emplean herramientas de análisis de datos como dashboards de control de calidad y tablas dinámicas de seguimiento, complementadas con logs de auditoría que registran eventos y excepciones. Finalmente, la tasa de uso por parte de la gerencia confirma si la solución adoptada cumple con los objetivos de usabilidad y valor estratégico.

Basado en lo anterior, la siguiente tabla presenta un conjunto de herramientas ampliamente utilizadas en la ingeniería de software para el análisis, la medición y el control de la calidad:

Tabla 2. Herramientas de análisis y métricas de calidad del software

Herramienta	Propósito principal	Licencia
SonarQube	Análisis estático de código, deuda técnica, calidad general.	Open Source / Comercial
JUnit / PyTest	Pruebas unitarias para Java y Python respectivamente.	Open Source
Selenium WebDriver	Automatización de pruebas funcionales de UI web.	Open Source
Apache JMeter	Pruebas de rendimiento, carga y estrés.	Open Source
OWASP ZAP	Pruebas de seguridad y detección de vulnerabilidades.	Open Source
JaCoCo / Coverage.py	Medición de cobertura de pruebas en Java y Python.	Open Source
Postman / Rest Assured	Pruebas funcionales de APIs REST.	Freemium / Open Source
Jenkins / GitHub Actions	Integración continua (CI/CD) y automatización de pipelines.	Open Source / Incluido
Jira	Seguimiento de defectos y gestión de proyectos ágiles.	Comercial
ESLint / Checkstyle	Verificación de estilo y convenciones de código.	Open Source

Herramienta	Propósito principal	Licencia
New Relic / Grafana	Monitoreo de rendimiento en producción.	Freemium / Open Source

1.3. Documentación de hallazgos

Documentar un hallazgo consiste en registrar formalmente cualquier anomalía, vulnerabilidad o área de mejora identificada durante la evaluación de la calidad. Esta práctica es fundamental para mitigar riesgos operativos y financieros.

- **Elementos esenciales de un reporte de hallazgos**

Para que un hallazgo sea útil, debe contener información y técnica precisa que permita a otros desarrolladores o analistas entender el problema, basado en lo siguiente:

- **Identificador único**

Código de referencia para el seguimiento (ej. ERR-001).

- **Severidad y prioridad**

Clasificación del impacto del error (crítico, mayor, menor).

- **Descripción del defecto**

Explicación clara de la desviación entre lo esperado y lo obtenido.

- **Pasos para la reproducción**

Secuencia lógica de acciones que disparan el error (algoritmo inverso).

- **Evidencia visual**

Capturas de pantalla o logs que demuestren el estado del error.

- **Clasificación de hallazgos según su origen**

En el análisis de software para diversas empresas del área, los hallazgos suelen clasificarse en tres categorías:

- **Hallazgos de datos (data quality)**

Inconsistencias en formatos, errores de tipeo o duplicidad de registros que afectan la integridad de la información.

- **Hallazgos de lógica (functional defects)**

Errores en fórmulas o macros que producen resultados incorrectos, como un cálculo de horas extra que excede la realidad.

- **Hallazgos de rendimiento (non-functional):**

Procesos que, aunque funcionan, son ineficientes o insostenibles ante grandes volúmenes de datos.

- **El informe de hallazgos como herramienta estratégica**

El producto final de la evaluación de calidad debe ser un informe técnico (en formato PDF para asegurar su integridad) que detalle el proceso de transformación de datos crudos a formatos óptimos. Este informe debe incluir:

- **Diagnóstico inicial**

Estado de la calidad antes de la intervención (calidad del dataset).

- **Estrategia de limpieza**

Acciones tomadas para normalizar la información (ej. concatenación, ajuste de formatos).

- **Validación de la solución**

Explicación de cómo se asegura que la automatización o el modelo funciona correctamente.

A continuación, se presenta la secuencia de estados que conforman la gestión de hallazgos, junto con las acciones que debe ejecutar el analista en cada etapa:

- **Identificado**

Se registra la inconsistencia en los datos o la macro.

- **En análisis**

Se utiliza el proceso de depuración para encontrar la causa raíz.

- **Corregido**

Se aplica la técnica de normalización o ajuste de código necesaria.

- **Verificado**

Se ejecuta la prueba de funcionalidad para validar el cierre.

Documentar correctamente los hallazgos permite "actuar como si le estuviera presentando su trabajo al gerente del área", demostrando no solo qué hace la solución, sino por qué es fiable y consistente.

2. Evaluación de requisitos no funcionales

Definen el "cómo" opera el sistema, a diferencia de los requisitos funcionales que dictan el "qué" hace. En el desarrollo de soluciones de software para la gestión estratégica de información, la calidad no es solo el resultado, sino la robustez de la arquitectura que lo soporta.

A continuación, se detallan los criterios técnicos para evaluar estos requisitos:

- **Seguridad**

La seguridad garantiza la protección de los datos contra accesos no autorizados o alteraciones accidentales. La gestión de la seguridad en el manejo de datos se fundamenta en garantizar la integridad y cifrado de los archivos fuente para evitar alteraciones durante la importación, junto con un estricto control de acceso que proteja la estructura de las soluciones y el código frente a manipulaciones de terceros. Estas medidas son pilares de una gestión de riesgos proactiva, orientada a identificar y mitigar posibles fallos de procesamiento o fugas de información que comprometan la operatividad y la reputación institucional.

- **Usabilidad**

Evalúa qué tan fácil es para el usuario final (como un gerente de operaciones) interactuar con la solución. La interfaz de usuario (UX) se optimiza mediante un panel de control con botones descriptivos y una visualización clara que utiliza gráficos adecuados y segmentadores (slicers) para filtrar datos intuitivamente. El objetivo principal es la simplicidad, permitiendo que el usuario final obtenga respuestas clave del negocio de forma directa y sin necesidad de conocimientos técnicos profundos.

- **Accesibilidad**

Asegura que la solución sea utilizable por el mayor número de personas, independientemente de sus capacidades técnicas. La estandarización mediante formatos consistentes y etiquetas claras en gráficos es fundamental para garantizar una lectura ágil de los indicadores en cualquier dashboard. Además, el diseño debe considerar el soporte multi-dispositivo, asegurando que las soluciones sean adaptables y funcionales tanto en herramientas de escritorio como en plataformas LMS y entornos virtuales de aprendizaje.

- **Portabilidad**

Se refiere a la capacidad de una solución de software para ejecutarse y mantenerse funcional en distintos entornos tecnológicos sin requerir modificaciones significativas. En el contexto de soluciones basadas en hojas de cálculo y automatización, este criterio implica garantizar la compatibilidad entre diferentes versiones del software, sistemas operativos y configuraciones de hardware. Para ello, se prioriza el uso de funciones estándar, la correcta gestión de rutas relativas, la eliminación de dependencias locales y la validación de macros en entornos controlados. Una solución portable reduce los riesgos asociados a la migración, facilita el despliegue en distintos equipos o sedes y asegura la continuidad operativa, incluso cuando la herramienta es transferida entre usuarios, áreas o plataformas institucionales.

- **Tiempos de respuesta**

Mide la eficiencia del sistema bajo condiciones de operación normal y picos de carga. La optimización de procesos busca una reducción drástica en los tiempos de ejecución, logrando transformar tareas manuales de varias horas en procesos de pocos

segundos, mediante la automatización. Esta eficiencia debe acompañarse de un alto rendimiento con grandes volúmenes de datos, asegurando la escalabilidad y velocidad de respuesta de las macros y modelos a medida que la información crece.

- **Adaptabilidad**

La adaptabilidad en el diseño de software es la capacidad de un sistema para evolucionar y ajustarse de manera eficiente a cambios en el entorno, los requisitos del usuario o las innovaciones tecnológicas sin comprometer su integridad. Un diseño adaptable se fundamenta en una arquitectura modular y el uso de patrones de diseño que reducen el acoplamiento, permitiendo que nuevas funcionalidades se integren con facilidad y que el software sea escalable ante el crecimiento de la demanda. Esta cualidad no solo extiende el ciclo de vida del producto, sino que también optimiza el mantenimiento y garantiza que la solución siga siendo funcional y competitiva en un mercado digital en constante transformación.

- **Confiabilidad**

La fiabilidad del sistema garantiza que la automatización realice sus funciones bajo condiciones específicas con una consistencia total, eliminando la variabilidad y los errores de copiado manual. Este nivel de precisión se alcanza mediante una rigurosa planificación y documentación de los pasos lógicos (algoritmos), lo que asegura resultados idénticos en cada ejecución y una estructura mucho más robusta que cualquier proceso basado en la improvisación.

- **Evaluación de hallazgos en RNF**

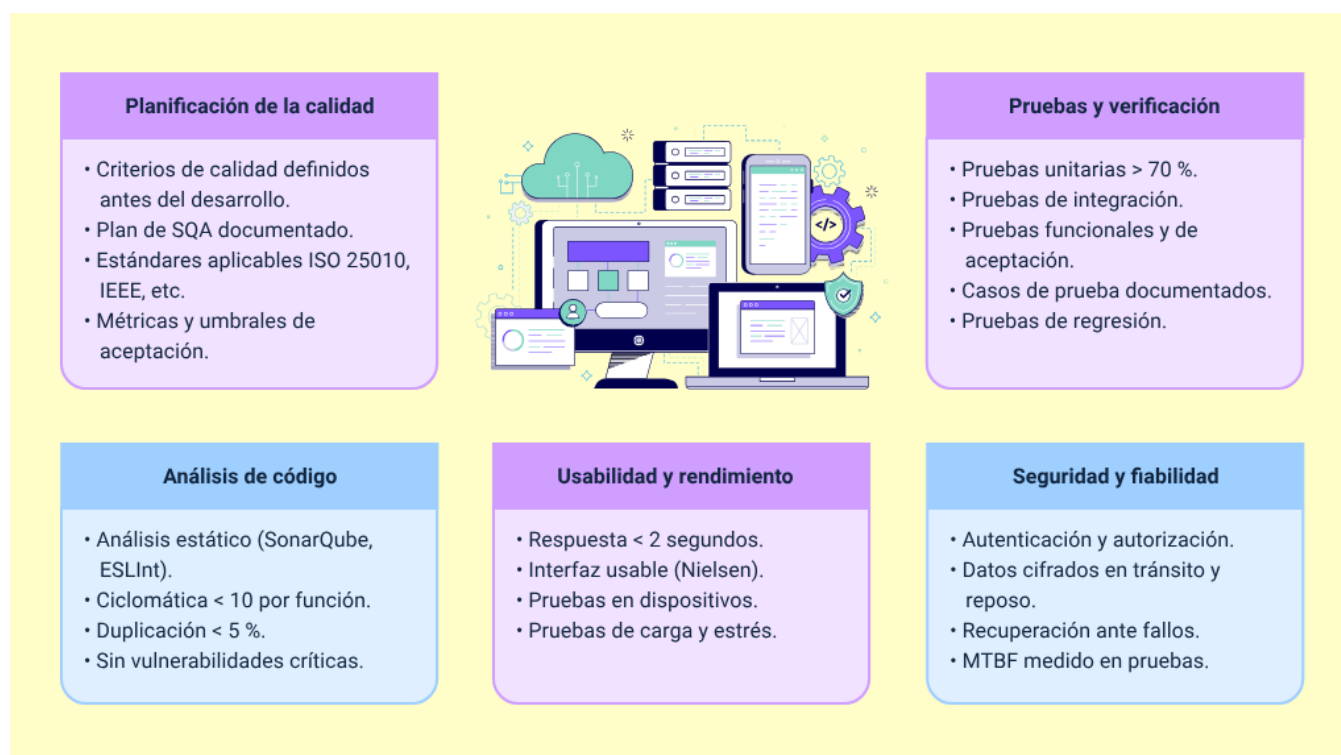
Al documentar hallazgos en estas áreas, el analista debe registrar no solo el error funcional, sino también la deficiencia en el rendimiento o la usabilidad. Por ejemplo, un

reporte que genera datos correctos, pero tarda 20 minutos en procesarse tiene un hallazgo crítico de tiempo de respuesta.

• Verificación de calidad

Marca cada criterio cumplido (☐) durante la evaluación o revisión del sistema y se representa de la siguiente manera:

Figura 1. Listado de verificación de calidad



Planificación de la calidad

- Criterios de calidad definidos antes del desarrollo.
- Plan de SQA documentado.
- Estándares aplicables ISO 25010, IEEE, etc.
- Métricas y umbrales de aceptación.

Análisis de código

- Análisis estático (SonarQube, ESLint).
- Ciclomática < 10 por función.
- Duplicación < 5 %.
- Sin vulnerabilidades críticas.

Usabilidad y rendimiento

- Respuesta < 2 segundos.
- Interfaz usable (Nielsen).
- Pruebas en dispositivos.
- Pruebas de carga y estrés.

Pruebas y verificación

- Pruebas unitarias > 70%.
- Pruebas de integración.
- Pruebas funcionales y de aceptación.
- Casos de prueba documentados.
- Pruebas de regresión.

Seguridad y fiabilidad

- Autenticación y autorización.
- Datos cifrados en tránsito y reposo.
- Recuperación ante fallos.
- MTBF medido en pruebas.

Interpretación:

90-100 % = Excelente calidad.

70-89 % = Buena calidad (revisar ítems pendientes).

50-69 % = Calidad aceptable (plan de mejora requerido).

< 50% = Calidad insuficiente (requiere trabajo significativo).

3. Gestión del conocimiento en el desarrollo de software

La Gestión del Conocimiento (GC) es el conjunto de procesos y prácticas para identificar, capturar, organizar, almacenar, compartir y aplicar el conocimiento generado en una organización. En proyectos de software, es clave para evitar repetir errores, acelerar el aprendizaje y mejorar continuamente la calidad del trabajo del equipo.

La gestión del conocimiento se divide principalmente en dos categorías:

A. Conocimiento tácito

Es personal, reside en las personas y se manifiesta a través de experiencias e intuiciones difíciles de formalizar.

B. Conocimiento explícito

Está debidamente documentado en manuales y bases de datos, lo que facilita su almacenamiento y distribución.

Esta distinción es clave para transformar las habilidades individuales en activos organizacionales que puedan ser compartidos y consultados por todo el equipo de trabajo.

Para que esta información sea útil, debe seguir un ciclo del conocimiento continuo que comienza con la creación y captura de nuevas experiencias, seguido de una organización estructurada que permita encontrarlas fácilmente. Finalmente, el proceso se completa al compartir, aplicar y evaluar dicho conocimiento, asegurando que se utilice para resolver problemas, tomar mejores decisiones y mejorar constantemente los procesos internos de la organización.

3.1. Modelos de gestión del conocimiento

La gestión del conocimiento se ha consolidado como un pilar estratégico para las organizaciones que buscan aprender, innovar y sostener ventajas competitivas en entornos complejos y cambiantes. Para comprender cómo el conocimiento se crea, circula y se aprovecha de manera sistemática, diversos autores han propuesto modelos conceptuales y operativos que orientan su gestión. Es así, como se presentan dos de los enfoques más reconocidos y complementarios:

A. Modelo SECI — Nonaka & Takeuchi (1995)

El modelo SECI es el marco teórico más influyente en gestión del conocimiento. Describe cómo el conocimiento se crea y transforma en las organizaciones a través de cuatro procesos de conversión. La dinámica no es lineal sino en espiral: cada ciclo genera conocimiento de mayor nivel de complejidad y valor. Cada una de sus fases se detalla de la siguiente manera:

Tabla 3. Modelo SECI

Fase del modelo	Descripción	Ejemplo en proyectos TI
Socialización (T→T)	Tácito a Tácito. Transferencia de conocimiento por observación, práctica y convivencia en equipo.	Programación en pareja, programación en grupo, mentorías técnicas, revisiones de código presenciales.
Externalización (T→E)	Tácito a Explícito. Convertir experiencias y saber-hacer en documentos formales.	Redactar lecciones aprendidas, crear wikis técnicas, documentar ADR, grabar tutoriales.
Combinación (E→E)	Explícito a Explícito. Integrar y sistematizar conocimiento ya documentado.	Construir bases de conocimiento, compilar manuales técnicos, crear plantillas estándar.

Fase del modelo	Descripción	Ejemplo en proyectos TI
Internalización (E→T)	Explícito a Tácito. Aprender haciendo a partir de documentación y casos documentados.	Seguir un manual hasta desarrollar habilidad práctica, estudiar proyectos anteriores.

B. Modelo de procesos organizacionales — Probst, Raub & Romhardt (2001)

Este modelo pragmático identifica siete procesos interconectados de gestión del conocimiento. A diferencia del SECI (más teórico), ofrece una guía operacional de qué hacer en cada etapa para gestionar el conocimiento organizacional de forma efectiva y se explica a continuación:

Tabla 4. Modelo de procesos organizacionales

#	Proceso	Descripción y actividades clave
01	Identificación	¿Qué conocimiento existe y qué falta? Mapas de conocimiento, inventarios de habilidades, análisis de brechas de competencias en el equipo.
02	Adquisición	Obtener conocimiento externo: capacitaciones, consultoría, benchmarking con otras organizaciones, incorporación de expertos.
03	Desarrollo	Crear nuevo conocimiento interno: I+D, prototipos, spikes de investigación en Scrum, laboratorios de innovación.
04	Distribución	Compartir con quien lo necesita: talleres, comunidades de práctica, tech talks, rotación planificada de roles y proyectos.
05	Utilización	Aplicar en el trabajo real: el conocimiento solo tiene valor cuando se usa. Indicador: frecuencia de consulta de la base de conocimiento.

#	Proceso	Descripción y actividades clave
06	Retención	Evitar pérdida de conocimiento crítico: documentación, entrevistas de salida, planes de sucesión, mentorías entre expertos y novatos.
07	Medición	Evaluar impacto: tiempo de onboarding, tasa de reuso, incidentes repetidos, satisfacción del equipo con los recursos disponibles.

El análisis conjunto de ambos modelos permite obtener una visión integral: comprender cómo emerge el conocimiento y, al mismo tiempo, cómo gestionarlo de forma efectiva dentro de las organizaciones.

3.2. Lecciones aprendidas en proyectos de software

Las lecciones aprendidas son el conocimiento adquirido durante la ejecución de un proyecto, tanto de los éxitos como de los errores y dificultades. Su correcta documentación y aplicación en proyectos futuros es uno de los activos más valiosos de una organización de software.

La estructura de una lección aprendida es:

- **Título descriptivo**

Qué ocurrió, breve y claro.

- **Área o categoría.**

Planificación, código, calidad, equipo, seguridad, etc.

- **Descripción del problema o situación**

Qué pasó exactamente.

- **Causas raíz identificadas**

Por qué ocurrió (técnica de los 5 Porqués).

- **Impacto en el proyecto**

Tiempo, costo, calidad, satisfacción del cliente.

- **Solución aplicada**

Qué se hizo para resolverlo.

- **Recomendación**

Qué hacer para evitarlo en proyectos futuros.

- **Responsable**

Del registro y fecha.

La siguiente tabla presenta un conjunto de lecciones aprendidas obtenidas de experiencias recurrentes en proyectos de desarrollo de software. Su sistematización permite identificar problemas frecuentes y establecer recomendaciones prácticas para mejorar la gestión y ejecución de futuros proyectos:

Tabla 5. Lecciones aprendidas recurrentes en proyectos de desarrollo de software

Lección Identificada	Área	Descripción	Recomendación
Falta de requisitos claros al inicio	Planificación	En el 60 % de proyectos con retrabajos costosos, los requisitos no estaban definidos ni validados con el cliente antes de comenzar el desarrollo.	Invertir tiempo en análisis de requisitos. Usar historias de usuario con criterios de aceptación (Given-When-Then) validadas por el cliente antes de codificar.
Estimaciones optimistas sin fundamento	Planificación	Las estimaciones basadas solo en la intuición sin datos históricos llevan a compromisos imposibles de cumplir y frustración del equipo y cliente.	Usar Planning Poker, T-shirt sizing o datos de velocidad de sprints anteriores. Aplicar el principio de triplicar la estimación inicial para tareas desconocidas.
No hacer pruebas durante el desarrollo	Calidad	Posponer pruebas para el final encontró defectos muy costosos. Los errores de integración no detectados a tiempo se multiplican exponencialmente.	Adoptar TDD o escribir pruebas unitarias en paralelo con el desarrollo. Integrar análisis estático en el IDE desde el primer commit.
Deuda técnica acumulada y no gestionada	Código	Proyectos que priorizaron velocidad sobre calidad sufrieron desaceleración severa en fases posteriores. Cada atajo se pagó con intereses compuestos.	Registrar deuda técnica en el backlog con impacto estimado. Dedicar al menos el 20 % de la capacidad de cada sprint a reducirla.
Comunicación deficiente en el equipo	Equipo	La falta de comunicación llevó a dos desarrolladores a implementar la misma funcionalidad de formas incompatibles, causando	Daily standups reales y eficientes. Canales de comunicación estructurados. Documentar decisiones técnicas importantes en registros ADR.

Lección Identificada	Área	Descripción	Recomendación
		conflictos de integración costosos.	
Ausencia de control de versiones adecuado	Código	Equipos sin estrategia de ramas definida sufrieron sobreescrituras de código, pérdida de trabajo y conflictos de merge imposibles de resolver.	Adoptar Git Flow o GitHub Flow desde el día uno. Definir política de commits descriptivos y code reviews obligatorios antes de cada merge.
No documentar mientras se desarrolla	Documentación	La documentación generada al final es incompleta. Los desarrolladores olvidan detalles de implementación y justificaciones de las decisiones tomadas.	Documentar en el mismo momento del desarrollo. Comentarios en código, README actualizado, ADR para decisiones arquitectónicas. Doc-as-Code en el repositorio.
Ignorar la seguridad durante el desarrollo	Seguridad	Agregar seguridad al final es exponencialmente más costoso. Las vulnerabilidades encontradas tarde requieren cambios arquitectónicos profundos.	Adoptar DevSecOps. Incluir Threat Modeling desde el diseño, validar <i>inputs</i> en cada capa y ejecutar OWASP ZAP en el pipeline CI/CD.

Práctica ágil: en Scrum, las lecciones aprendidas se capturan en la retrospectiva del Sprint. La dinámica 'Start / Stop / Continue' es una de las más efectivas: ¿Qué se debe empezar a hacer? ¿Qué se debe dejar de hacer? ¿Qué se está haciendo bien y se debe continuar?

3.3. Recomendaciones de buenas prácticas

En el siguiente pódcast se abordarán buenas prácticas clave en el desarrollo de software, orientadas a fortalecer la calidad, el aprendizaje continuo y la mejora de los procesos. A partir de ejemplos prácticos, se analizará cómo estas prácticas contribuyen a construir soluciones más confiables, sostenibles y alineadas con las necesidades de los usuarios y de la organización:

Transcripción del pódcast: Recomendaciones de buenas prácticas

Hombre: en desarrollo de software hay equipos que pasan meses resolviendo problemas complejos, ajustando procesos, entendiendo por qué ciertas cosas fallan, y con el tiempo descubren que muchas de esas soluciones quedan repartidas entre reuniones, chats, comentarios sueltos o simplemente en la memoria de quienes participaron en ese momento.

Mujer: la mayoría de los equipos nota eso cuando llega alguien nuevo al proyecto y empiezan otra vez las preguntas sobre configuraciones, decisiones técnicas o procesos que todos parecían tener claros unos meses atrás, pero después quedaron dispersos entre conversaciones, tareas del día a día y cosas que nadie volvió a registrar.

Hombre: cuando un proyecto empieza a crecer, también empiezan a aparecer decisiones, configuraciones, cambios y formas de trabajar que el equipo necesita recordar después para no perder tiempo reconstruyendo cosas que ya había entendido antes.

Mujer: por eso en muchos equipos la documentación empieza a construirse casi al mismo tiempo que el código. Mientras aparecen nuevas funcionalidades también se actualizan README, se registran configuraciones, se documentan cambios importantes en Git o se escriben ADR, los Architecture Decision Records, donde el equipo va dejando registro de decisiones que más adelante ayudan a entender porque el proyecto tomó ciertos caminos.

Hombre: con el tiempo, esos registros empiezan a resolver preguntas que al principio nadie imaginaba. Hay decisiones que durante una reunión parecen completamente obvias, pero meses después necesitan contexto para recordar qué estaba pasando en ese momento, qué problema intentaban resolver o por qué el equipo terminó inclinándose por determinada solución.

Mujer: cuando aparece un incidente importante dentro del sistema, muchas veces el análisis técnico empieza a revelar pequeños detalles que durante el trabajo diario habían pasado desapercibidas. Una dependencia mal configurada, un cambio apresurado, información que no circuló bien o procesos que distintas personas estaban interpretando de maneras diferentes.

Hombre: en varios equipos, las retrospectivas y los post-mortems permiten reconstruir con más calma todo lo que ocurrió alrededor del incidente. Ahí empiezan a aparecer patrones, decisiones repetidas, puntos débiles en algunos procesos y detalles que probablemente habrían vuelto a generar problemas más adelante.

Mujer: dentro de los proyectos existe mucho conocimiento que rara vez queda bien documentado. Son prácticas que el equipo aprende trabajando todos los días,

formas de desplegar servicios, maneras de resolver incidentes, pequeños criterios técnicos o incluso atajos que terminan facilitando tareas complejas.

Hombre: dinámicas como el pair programming o las comunidades de práctica ayudan a que muchas decisiones técnicas circulen de manera mucho más natural dentro del equipo. Mientras el trabajo avanza, también empieza a compartirse experiencia, contexto y formas de resolver problemas que normalmente quedarían concentradas en unas pocas personas.

Mujer: algo parecido pasa con los screencasts internos o las wikis colaborativas. A veces una explicación corta grabada por alguien del equipo termina resolviendo dudas que probablemente después aparecerían repetidas en reuniones, mensajes o sesiones de soporte improvisadas.

Hombre: cuando toda esa información empieza a estar organizada, el proceso de onboarding se vuelve mucho más fluido. Las personas nuevas logran entender más rápido cómo funciona el proyecto, dónde consultar procesos, cómo están estructurados los servicios y qué dinámicas hacen parte del trabajo diario del equipo.

Mujer: con el tiempo, toda esa documentación también empieza a mostrar cosas interesantes sobre la forma en que trabaja el equipo. Hay procesos que generan dudas constantemente, decisiones que necesitan demasiadas aclaraciones o errores que vuelven a aparecer incluso después de haberse solucionado varias veces.

Hombre: con el tiempo, toda esa información empieza a influir directamente en la forma en que el equipo mejora sus procesos. Las decisiones tienen más contexto, muchos errores dejan aprendizajes visibles y el proyecto empieza a sostenerse sobre

conocimiento que permanece disponible incluso cuando cambian las personas o evolucionan las tecnologías.

Mujer: poco a poco el equipo empieza a construir trazabilidad sobre muchas decisiones y conserva aprendizajes que más adelante vuelven a ser útiles cuando el proyecto entra en nuevas etapas o aparecen retos similares.

Hombre: en desarrollo de software, buena parte de la experiencia del equipo nace en situaciones cotidianas que al principio parecen pequeñas. Un incidente, una configuración difícil, una decisión técnica tomada bajo presión o incluso un error que obligó a revisar algo que nadie había cuestionado antes.

Mujer: cuando todo eso logra mantenerse visible y disponible para el equipo, el proyecto empieza a construir memoria, aprendizaje colectivo y una forma de trabajo que sigue fortaleciéndose mientras el sistema y las personas continúan evolucionando.

Hombre: Gracias por su compañía. Nos escuchamos en el próximo programa.

3.4. Herramientas para la gestión del conocimiento

Existen herramientas especializadas que facilitan la captura, organización y distribución del conocimiento en equipos de software. La mayoría son gratuitas o tienen planes accesibles para estudiantes y equipos pequeños. A continuación, una breve explicación de cada una:

Tabla 6. Herramientas del conocimiento

Herramienta	Descripción	Categoría	Licencia
Confluence	Wiki empresarial. Integración nativa con Jira y toda la suite Atlassian.	Bases de conocimiento	Empresarial
Notion	Wiki flexible y colaborativa, popular en <i>startups</i> . Versión gratuita generosa.	Bases de conocimiento	Freemium
GitHub Wiki	Wiki integrada en repositorios GitHub. Ideal para proyectos de código abierto.	Doc-as-Code	Open Source
Obsidian	Gestión de notas con Markdown y enlaces bidireccionales. Ideal para conocimiento personal.	Bases de Conocimiento	Freemium
Miro	Pizarra colaborativa virtual para retrospectivas y mapas de conocimiento.	Retrospectivas	Freemium
EasyRetro	Herramienta específica para retrospectivas ágiles de equipo. Simple y efectiva.	Retrospectivas	Freemium
Slack / Teams	Mensajería con canales temáticos de conocimiento, tips técnicos y recursos.	Comunicación	Freemium
Loom	Grabación de screencasts y videotutoriales asíncronos. Captura conocimiento visual.	Captura tácito	Freemium

Herramienta	Descripción	Categoría	Licencia
Moodle	LMS Open Source. Estructura conocimiento en cursos y módulos.	Formación	Open Source
Markdown + Git	Documentación en texto plano versionada junto al código. Práctica estándar.	Doc-as-Code	Open Source
MkDocs / Docusaurus	Generadores de sitios de documentación a partir de Markdown con búsqueda integrada.	Doc-as-Code	Open Source
ADR Tools	Gestión de Architecture Decision Records. Registros de decisiones técnicas versionados.	Decisiones	Open Source

4. Planes de mejora

El plan de mejora es un instrumento de gestión que formaliza acciones, responsables y plazos para cerrar brechas de calidad, permitiendo que los hallazgos de las evaluaciones se traduzcan en respuestas operativas reales. Basado en normativas como la ISO 9001:2015 y el Ciclo PHVA de Deming (Planear, Hacer, Verificar, Actuar), este proceso es fundamental para la mejora continua y el alto desempeño en ingeniería de software, alineándose con marcos de trabajo como CMMI v2.0 e ISO/IEC 15504 (SPICE) para elevar la madurez de los procesos y productos del equipo.

Los siguientes son los tipos de acciones en un plan de mejora:

- **Correctivas**

Se orientan a eliminar la causa raíz de problemas que ya se han presentado. Buscan evitar que las no conformidades, fallos o desviaciones se repitan, mediante el análisis sistemático de lo ocurrido y la implementación de soluciones sostenibles.

- **Preventivas**

Están dirigidas a identificar y eliminar las causas de problemas potenciales antes de que ocurran. Se basan en la anticipación de riesgos, el análisis de escenarios y la aplicación de controles que reduzcan la probabilidad de fallos futuros.

- **De mejoramiento**

Tienen como objetivo incrementar el desempeño de procesos, productos o servicios más allá de los requisitos mínimos establecidos. Aprovechan oportunidades de optimización, innovación y aprendizaje continuo para generar mayor valor organizacional.

Clave: un plan de mejora sin indicadores de verificación y sin responsables nominados, es solo una lista de buenos deseos. La diferencia entre un plan efectivo y uno inútil está en la concreción, el seguimiento y el compromiso real de los involucrados.

4.1. Elaboración del plan de mejora

La elaboración es la fase de diseño del plan. Comprende identificar las brechas, analizar sus causas raíz y estructurar las acciones de respuesta con todos sus atributos. Un plan bien elaborado hace que la ejecución y el seguimiento sean viables y efectivos.

- **Pasos para elaborar el plan**

A continuación, se describen los pasos para elaborar un plan de mejora de forma estructurada y sistemática. Estos pasos permiten transformar hallazgos y brechas identificadas en acciones concretas, medibles y asignables, facilitando su ejecución, seguimiento y evaluación de resultados:

- **Paso 1. Identificar la brecha o no conformidad**

Partir de los resultados de una evaluación: auditoría, retrospectiva, revisión de métricas o queja del cliente. Describir el hallazgo con precisión y evidencia objetiva.

- **Paso 2. Analizar la causa raíz**

Aplicar técnicas de análisis causal: técnica de los 5 Porqués, diagrama de Ishikawa (espina de pescado), FMEA o árbol de fallas. No actuar sobre síntomas sino sobre causas sistémicas.

- **Paso 3. Definir el objetivo de mejora SMART**

Específico, medible, alcanzable, relevante y con Tiempo definido. Ejemplo
“Reducir densidad de defectos de 3 a 0.8 bugs/KLOC en 3 meses”.

- **Paso 4. Proponer acciones concretas**

Para cada causa raíz, definir al menos una acción. Usar verbos en infinitivo: implementar, capacitar, automatizar, establecer.

- **Paso 5. Asignar responsables y recursos**

Nominar un Accountable (A) y un Responsable (R) para cada acción. Estimar recursos: tiempo, herramientas, capacitación, presupuesto.

- **Paso 6. Establecer indicadores y fechas**

Definir el KPI que medirá el éxito y la fecha límite de cumplimiento. El indicador debe ser verificable con datos objetivos.

- **Paso 7. Documentar y socializar**

El plan debe ser aprobado por el Accountable (A), socializado con todos los involucrados y almacenado en un lugar accesible. Un plan que nadie conoce no se ejecuta.

La siguiente tabla presenta una plantilla estándar para la elaboración y gestión del plan de mejora. Su estructura permite registrar de forma ordenada cada acción, asegurar la trazabilidad entre hallazgos, causas y soluciones, y facilitar el seguimiento, control y evaluación de resultados mediante responsables, indicadores y evidencias objetivas:

Tabla 7. Plantilla estándar del plan de mejora

Campo	Descripción	Ejemplo
ID de la acción	Identificador único.	AC-001, AP-003, AM-007.
Hallazgo / Brecha	No conformidad o brecha identificada.	Cobertura de pruebas en 35 % (meta: 70 %).
Causa raíz	Resultado del análisis causal (5 porqués).	Sin política de pruebas en el proceso.
Tipo de acción	Correctiva / Preventiva / Mejoramiento.	Correctiva.
Acción propuesta	Qué se va a hacer, en concreto.	Implementar TDD como práctica obligatoria.
Responsable (R)	Quién ejecuta la acción.	Desarrollador líder del módulo.
Accountable (A)	Quién responde por el resultado.	Líder de proyecto.
Consulted (C)	A quién se consulta.	Arquitecto de software.
Informed (I)	A quién se informa.	Cliente, dirección del proyecto.
Fecha de inicio	Fecha programada de inicio.	2025-03-01.
Fecha límite	Fecha máxima de cierre.	2025-04-30.
Recursos	Recursos necesarios para ejecutar.	8 horas de capacitación, SonarQube.
Indicador de éxito	KPI que confirma el cumplimiento.	Cobertura ≥ 70 % en sprint review.
Estado	Semáforo de avance actual.	En progreso.
Evidencia de cierre	Dato que demuestra cumplimiento.	Reporte JaCoCo con cobertura 74 %.

- **Técnica de los 5 Porqués — Ejemplo aplicado**

A continuación, se presenta un ejemplo aplicado de la Técnica de los 5 Porqués, con el fin de ilustrar su uso práctico en la identificación de la causa raíz de un problema recurrente en proyectos de software y la definición de una acción correctiva concreta y accionable:

- **Problema**

Alta tasa de defectos en producción.

- ¿Por qué hay defectos en producción? Porque el código no fue probado suficientemente antes del despliegue.
- ¿Por qué no fue probado? Porque no había tiempo, los sprints se comprimieron.
- ¿Por qué se comprimieron? Porque las estimaciones iniciales fueron muy optimistas.
- ¿Por qué fueron optimistas? Porque no se usaron datos históricos de velocidad del equipo.
- ¿Por qué no se usaron datos históricos? Porque no existía un registro sistemático de métricas de velocidad. ← Causa raíz

- **Acción correctiva resultante**

Implementar tablero de métricas de velocidad en Jira y usarlo obligatoriamente en la planificación de cada sprint.

- Responsable: Scrum master.
- Fecha límite: próxima planificación de sprint.

- **Error frecuente**

Confundir síntomas con causas raíz.

- “El sistema tiene errores” es un síntoma.
- “No hay proceso formal de revisión de código antes del merge” es una causa raíz accionable.
- El plan debe atacar siempre la causa raíz.

4.2. Acciones correctivas

Las acciones correctivas son medidas esenciales que se aplican tras la detección de un defecto o falla, exigiendo un análisis de causa raíz (ACR) obligatorio para evitar soluciones superficiales. A diferencia de una simple corrección puntual, estas acciones buscan eliminar el origen del problema y requieren una verificación de eficacia posterior para confirmar que la incidencia no se repita. Su implementación garantiza que se modifiquen los procesos necesarios para impedir que fallos similares vuelvan a afectar la producción en el futuro.

Basado en lo anterior, la siguiente tabla contiene acciones aplicadas a ejemplos reales:

Tabla 8. Ejemplos de acciones correctivas en proyectos de software

ID	Problema	Causa raíz	Acción correctiva	Impacto	Plazo
AC-001	Alta tasa de defectos en producción (5 bugs/KLOC).	Sin proceso formal de revisión de código ni pruebas automatizadas.	Implementar code review obligatorio (mínimo 1 revisor) y pipeline CI con pruebas unitarias antes	Alta	30 días

ID	Problema	Causa raíz	Acción correctiva	Impacto	Plazo
			de cada merge a rama principal.		
AC-002	Retrasos recurrentes en entregas de sprint.	Estimaciones sin base histórica — no se usaban datos de velocidad real del equipo.	Implementar tablero de métricas de velocidad en Jira y usar Planning Poker con datos históricos en cada planificación de sprint.	Alta	Próximo sprint
AC-003	Vulnerabilidades de seguridad críticas en producción.	Sin análisis de seguridad integrado en el pipeline de desarrollo.	Integrar OWASP ZAP y análisis estático de seguridad (SonarQube) en el CI/CD. Sin green en seguridad, no hay deploy a producción.	Crítica	15 días
AC-004	Pérdida de código en conflictos de merge.	Sin estrategia de ramas definida ni política de commits del equipo.	Adoptar GitHub Flow: main protegido + feature branches + pull requests obligatorios. Capacitar al equipo en sesión de 2 horas.	Media	1 semana
AC-005	Documentación técnica inexistente al cierre.	La documentación no era parte del proceso ni del Definition of Done.	Actualizar Definition of Done: ninguna historia se cierra sin README actualizado y ADR si hubo decisión arquitectónica.	Media	Próximo sprint

Se detalla una diferencia clave:

- **Corrección**

Resuelve el problema puntual “se arregló el bug”.

- **Acción correctiva**

Elimina lo que causó el bug sistemáticamente “se implementó revisión de código obligatoria y pruebas de regresión automatizadas para evitar que este tipo de error vuelva a llegar a producción”.

4.3. Acciones preventivas

Las acciones preventivas eliminan causas de no conformidades potenciales, antes de que ocurran. Se basan en análisis de riesgo, lecciones aprendidas de proyectos anteriores y tendencias de indicadores que aún no son críticos, pero muestran deterioro.

- **Acciones de mejoramiento**

Las acciones de mejoramiento van más allá del cumplimiento mínimo, ya que buscan innovar, automatizar, optimizar y elevar el estándar de desempeño; además, no responden a un problema sino a una oportunidad identificada para ser más eficientes o eficaces.

Con el fin de clarificar el rol de las acciones de mejoramiento dentro de un plan de mejora, a continuación, se presentan dos tablas de apoyo. La primera compara las acciones correctivas, preventivas y de mejoramiento, según distintas dimensiones clave; mientras que la segunda ilustra ejemplos concretos de acciones preventivas y de mejoramiento aplicadas en contextos reales de proyectos de software:

Tabla 9. Comparativo acciones correctivas, preventiva y de mejoramiento

Dimensión	Correctiva	Preventiva	Mejoramiento
Momento de aplicación	Después del problema ocurrido.	Antes de que el riesgo ocurra.	Sin problema identificado.
Disparador	No conformidad o defecto real.	Riesgo o tendencia negativa detectada.	Oportunidad de optimización.
Base de análisis	Causa raíz del problema ocurrido.	Análisis de riesgos y lecciones aprendidas.	<i>Benchmarking</i> y mejores prácticas.
Objetivo	Eliminar causa para que no se repita.	Evitar que el riesgo se materialice.	Elevar el nivel de desempeño.
Urgencia	Alta (ya hay impacto real).	Media (riesgo latente).	Baja-media (inversión en mejora).
Ejemplo concreto	Automatizar pruebas tras fallo en producción.	Checklist de seguridad pre-lanzamiento.	Implementar DevOps para acelerar CI/CD.

Tabla 10. Ejemplos de acciones preventivas y de mejoramiento

Tipo	ID	Situación	Acción propuesta	Beneficio esperado
Preventiva	AP-001	El líder técnico único concentra todo el conocimiento del sistema.	Programa de rotación de roles y pair programming semanal para distribuir conocimiento entre al menos 3 integrantes del equipo.	Elimina el bus factor — el equipo no se paraliza si alguien sale.
Preventiva	AP-002	Complejidad ciclomática, subió de 6 a 9 en 2	Regla en SonarQube: build failed si función supera CC=10.	Previene código imposible de mantener antes de

Tipo	ID	Situación	Acción propuesta	Beneficio esperado
		sprints (tendencia negativa).	Refactorizar funciones actuales con CC ≥ 10 .	que sea deuda crítica.
Preventiva	AP-003	Sin plan de continuidad ante falla del servidor de producción.	Implementar backup automático diario y proceso de recuperación ante desastres (RTO < 4 horas). Documentar y probar el plan.	Garantiza disponibilidad ante incidentes graves.
Mejoramiento	AM-001	El despliegue manual toma 3 horas y genera errores humanos frecuentes.	Pipeline CI/CD completo con GitHub Actions: despliegue automático a staging en cada merge + producción con aprobación manual.	Reduce despliegue de 3 horas a 8 minutos, elimina errores manuales.
Mejoramiento	AM-002	El equipo no tiene visibilidad del estado de calidad del proyecto en tiempo real.	Dashboard de métricas en SonarQube + tablero Kanban público en Jira visible para equipo y stakeholders.	Aumenta transparencia, agiliza decisiones técnicas.
Mejoramiento	AM-003	Las retrospectivas no generan mejoras por falta de seguimiento de compromisos.	Tablero de “Compromisos de Retrospectiva” en Notion: cada ítem con responsable, fecha y evidencia. Revisión en el siguiente sprint review.	Convierte retrospectivas en motor real de mejora continua.

Fuentes para identificar preventivas y mejoramiento: análisis de riesgos (FMEA, matriz de riesgos), benchmarking con la industria, sugerencias del equipo en retrospectivas, auditorías internas proactivas y revisión de tendencias de métricas de calidad.

4.4. Responsables del plan de mejora

Asignar responsables con claridad es uno de los factores más críticos para el éxito de un plan de mejora. Toda acción sin un responsable nominado tiene probabilidad cercana a cero de ejecutarse. La matriz RACI define con precisión quién hace qué (roles) de la siguiente manera:

- **R — Responsible**

Quien ejecuta la acción directamente. Puede haber más de uno. Debe tener las competencias técnicas y el tiempo disponible.

- **A — Accountable**

Quien responde por el resultado final. Solo puede haber uno por acción. Aprueba el trabajo del Responsable y rinde cuentas ante la dirección.

- **C — Consulted**

A quien se consulta antes o durante la ejecución. Provee criterios especializados. Comunicación bidireccional.

- **I — Informed**

A quien se informa del avance y resultados. Comunicación unidireccional. No participa en la ejecución.

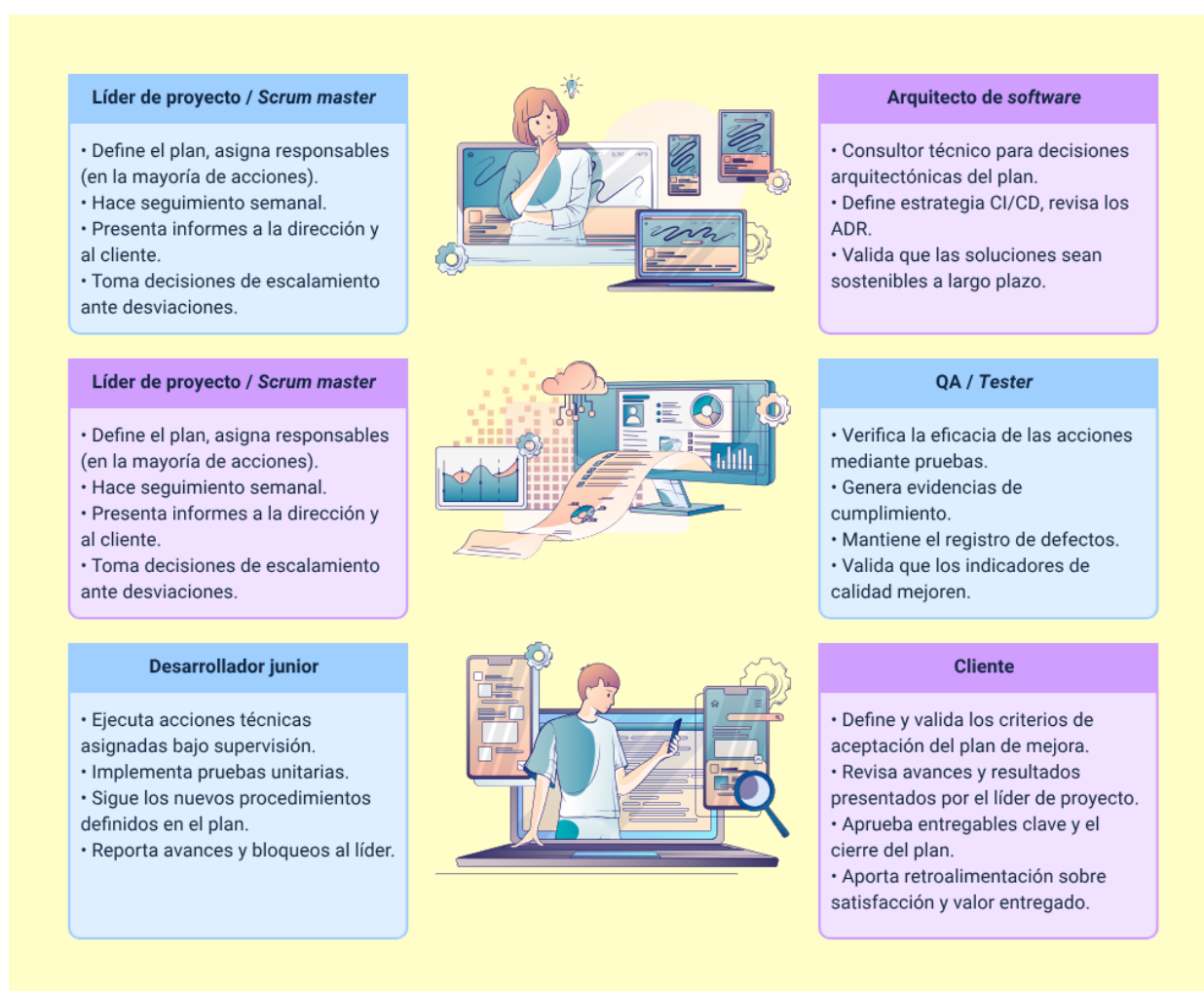
Con base en la definición de roles establecida por la matriz RACI, la siguiente tabla muestra la asignación de responsabilidades para las principales actividades del plan de mejora de software. Esta matriz permite visualizar de manera explícita quién ejecuta, quién responde, a quién se consulta y a quién se informa en cada acción, reduciendo ambigüedades y fortaleciendo la gobernanza del plan:

Tabla 11. Matriz RACI — Plan de mejora de software

Actividad / Acción	Líder proyecto	Desarrollador senior	Desarrollador junior	QA / Tester	Arquitecto de software	Cliente
Elaborar plan de mejora.	A	R	C	C	C	I
Análisis de causa raíz.	A	R	C	R	C	I
Ejecutar acción correctiva.	A	R	R	C	C	I
Implementar pruebas automatizadas.	C	A	R	R	C	I
Revisar código (code review).	I	A	R	C	C	—
Ejecutar pruebas de regresión.	C	C	R	A	I	I
Configurar pipeline CI/CD.	C	A	R	C	R	I
Verificar eficacia de acciones.	A	R	C	R	I	C
Generar informe de avance.	A	R	I	I	I	C
Aprobar cierre del plan.	A	C	I	I	C	R

Con el fin de profundizar en las responsabilidades asociadas a cada rol del plan de mejora, la siguiente imagen describe de manera sintética las funciones clave de cada uno. Esta descripción permite comprender cómo cada rol contribuye de forma específica a la planificación, ejecución, seguimiento y validación de las acciones de mejora definidas:

Figura 2. Perfiles de responsables en proyectos desarrollo de software



Líder de proyecto / Scrum master

- Define el plan, asigna responsables (en la mayoría de acciones).
- Hace seguimiento semanal.
- Presenta informes a la dirección y al cliente.
- Toma decisiones de escalamiento ante desviaciones.

Arquitecto de software

- Consultor técnico para decisiones arquitectónicas del plan.
- Define estrategia CI/CD, revisa los ADR.
- Valida que las soluciones sean sostenibles a largo plazo.

Desarrollador senior / Tech lead

- Ejecuta las acciones técnicas más complejas.
- Lidera el análisis de causa raíz, revisa el trabajo técnico de otros.
- Define estándares de codificación y arquitectura del plan.

QA / Tester

- Verifica la eficacia de las acciones mediante pruebas.
- Genera evidencias de cumplimiento.
- Mantiene el registro de defectos.
- Valida que los indicadores de calidad mejoren.

Desarrollador junior

- Ejecuta acciones técnicas asignadas bajo supervisión.

- Implementa pruebas unitarias.
- Sigue los nuevos procedimientos definidos en el plan.
- Reporta avances y bloqueos al líder.

Cliente

- Define y valida los criterios de aceptación del plan de mejora.
- Revisa avances y resultados presentados por el líder de proyecto.
- Aprueba entregables clave y el cierre del plan.
- Aporta retroalimentación sobre satisfacción y valor entregado.

Regla de oro RACI: una acción debe tener exactamente un Accountable (A). Si hay más de uno, la responsabilidad se diluye y en la práctica nadie se siente verdaderamente responsable del resultado final.

4.5. Verificación y seguimiento

La verificación comprueba que las acciones se ejecutaron y que produjeron el resultado esperado. El seguimiento es el proceso continuo de monitoreo del avance. Sin esta fase, el plan de mejora es solo papel: los resultados reales solo existen si se miden y documentan con evidencia objetiva.

Los indicadores clave de seguimiento (KPIs) del plan establecen metas rigurosas para garantizar la calidad, exigiendo que al menos el 90 % de las acciones se completen a tiempo y que la densidad de fallos sea inferior a 1 defecto/KLOC. La eficacia técnica se mide mediante una cobertura de pruebas unitarias ≥ 70 % con JaCoCo, una duplicación de código < 5 % en SonarQube y la erradicación total de la recurrencia en no conformidades críticas. Finalmente, el cumplimiento administrativo requiere que el 100

% de las acciones cuente con su respectiva evidencia de cierre documentada, asegurando un proceso de mejora totalmente trazable y exitoso.

Estados de avance o escala de semáforo, se presenta la siguiente imagen que detalla el estado de cada acción, junto a su criterio y la acción requerida en cada uno:

- **Completada**
 - ejecutada, evidencia disponible, indicador cumplido.
Documentar y cerrar. Estandarizar si aplica.
- **En progreso**
 - Iniciada, dentro del plazo, avance $\geq 50\%$.
Continuar. Verificar recursos disponibles.
- **En riesgo**
 - Avance $< 50\%$ con menos del 30 % del tiempo restante.
Escarlar al Accountable (A). Revisar plan y recursos.
- **Vencida**
 - Plazo superado sin completar la acción.
Análisis inmediato. Reprogramar con justificación.
- **Cancelada**
 - No viable o el contexto cambió significativamente.
Documentar motivo. Evaluar acción alternativa.

- **Tipos de evidencias de verificación**

Para garantizar la trazabilidad de un plan de mejora, es fundamental recopilar evidencias técnicas y administrativas sólidas. Estas incluyen los reportes de herramientas como SonarQube para deuda técnica o JaCoCo para cobertura, junto con

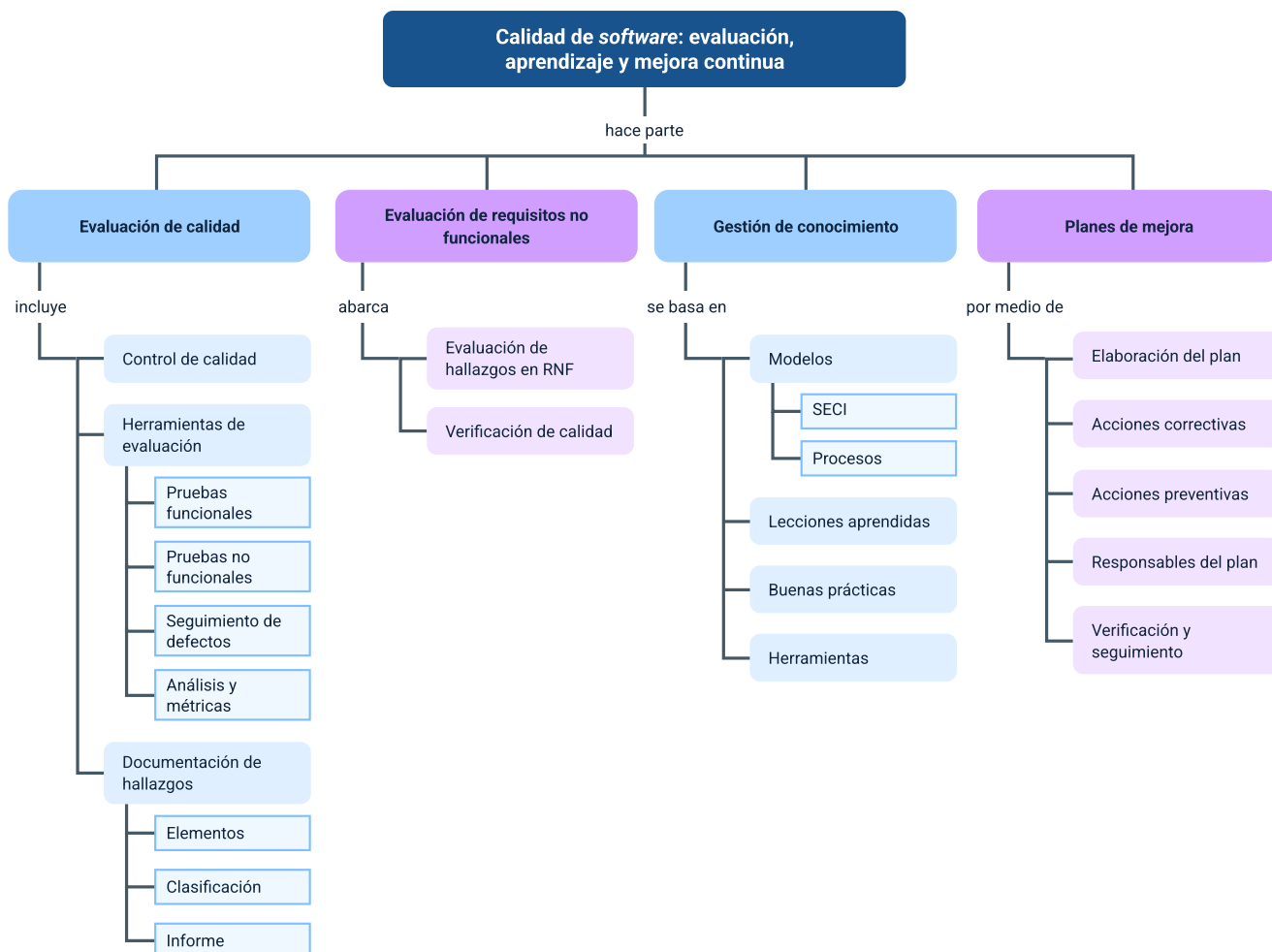
capturas de pantalla de los dashboards CI/CD en verde y dashboards de métricas que demuestren un impacto medible. Asimismo, se deben adjuntar actas de reuniones, checklists de verificación firmados por el líder de QA y los enlaces a los Pull Requests en GitHub o GitLab, los cuales sirven como prueba directa de la implementación técnica y su respectiva revisión de código.

El éxito de estas acciones depende de una frecuencia de seguimiento rigurosa: las tareas críticas requieren un control semanal, mientras que las de impacto medio se revisan de forma quincenal. Mensualmente, se debe realizar una revisión formal del plan completo con un informe escrito dirigido a la dirección, culminando con una verificación de eficacia final al cierre de cada actividad. Este proceso asegura que cada mejora no solo se ejecute, sino que cumpla con el indicador de éxito establecido y esté debidamente documentada en plataformas como Jira o Confluence.

Buena práctica: el seguimiento tardío convierte problemas solucionables en incumplimientos definitivos. Detectar que una acción está “En riesgo” con 3 semanas de anticipación, permite tomar medidas correctivas. Detectarlo el día del vencimiento no permite ninguna acción.

Síntesis

El componente presenta una guía técnica integral sobre la calidad del software, estructurada desde la evaluación sistemática y el control de procesos, hasta la consolidación de una cultura de mejora continua. A través de estándares como la serie ISO/IEC 25000, se profundiza en el uso de herramientas para pruebas funcionales y no funcionales (seguridad, usabilidad y desempeño), la documentación rigurosa de hallazgos y el seguimiento de defectos, mediante procesos de depuración. Finalmente, integra la gestión del conocimiento por medio de modelos como el SECI para capitalizar lecciones aprendidas y establece la metodología para elaborar planes de mejora efectivos basados en acciones correctivas y preventivas, asegurando la sostenibilidad y excelencia del servicio de software.



Glosario

Algoritmo: secuencia lógica y finita de pasos bien definidos que resuelven un problema o realizan una tarea específica.

Aseguramiento de la calidad (QA): proceso proactivo y preventivo orientado a los procesos de desarrollo para evitar la aparición de defectos mediante estándares.

Calidad del software: grado en que un sistema satisface los requisitos especificados y las necesidades implícitas del usuario.

Caso de prueba: conjunto de condiciones o variables documentadas con resultados esperados para determinar si el software funciona correctamente.

Causa raíz: origen sistémico de un problema identificado mediante técnicas como los "5 Porqués", cuya eliminación evita la repetición del error.

Conocimiento explícito: saber formalizado y documentado en manuales, procedimientos o lecciones aprendidas, fácil de compartir y almacenar.

Conocimiento tácito: saber personal y difícil de formalizar, basado en experiencias, intuiciones y habilidades prácticas de los individuos.

Control de calidad (SQC): proceso sistemático de verificar que el producto final (reporte, dashboard o macro) cumpla con las especificaciones técnicas.

Depuración (Debugging): conjunto de técnicas para identificar, analizar y corregir errores o bugs en el código de un programa.

Hallazgo: registro formal de cualquier anomalía, vulnerabilidad o área de mejora identificada durante una evaluación de calidad.

Lecciones aprendidas: conocimiento adquirido de éxitos y errores durante un proyecto, sistematizado para mejorar trabajos futuros.

Métrica: dato objetivo y cuantificable utilizado para evaluar la eficacia del desarrollo y la calidad del producto final.

No conformidad: incumplimiento de un requisito preestablecido que activa la necesidad de acciones correctivas en un plan de mejora.

Plan de mejora: instrumento de gestión que formaliza acciones, responsables y plazos para superar brechas de calidad.

Usabilidad: atributo que evalúa la facilidad con la que el usuario final interactúa con la solución de software.

Referencias bibliográficas

Alexander, M. & Kusleika, D. (2019). Excel 2019 power programming with VBA. John Wiley & Sons.

International Organization for Standardization. (2015). Quality management systems — Requirements (ISO 9001:2015).

International Organization for Standardization. (2023). Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models (ISO/IEC 25010:2023).

Jelen, B. (2020). Excel Dynamic Arrays Straight to the Point (2nd ed.). Tickling Keys, Inc.

Jelen, B. & Syrstad, T. (2022). Microsoft Excel VBA and Macros (Office 2021 and Microsoft 365). Microsoft Press.

McFedries, P. (2019). Microsoft Excel 2019 formulas and functions. Microsoft Press.

Nonaka, I. & Takeuchi, H. (1995). The knowledge-creating company: How Japanese companies create the dynamics of innovation. Oxford University Press.

Probst, G., Raub, S. & Romhardt, K. (2001). Gestión del conocimiento: los componentes del éxito. Prentice Hall.

Winston, W. (2016). Microsoft Excel data analysis and business modeling. Microsoft Press.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico – Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Solanlly Sánchez Melo	Experta temática	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Juan Daniel Polanco Muñoz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Veimar Celis Meléndez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Francisco José Vásquez Suárez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Ernesto Navarro Jaimes	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluadora de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima

Nombre	Cargo	Centro de Formación y Regional
Javier Mauricio Oviedo	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima